Hand-in Exercise 2

Deadline: March 20, 13:15

Read these instructions carefully before you start coding.

This first hand-in exercise covers material from the **lectures 4 through 6** (up to and including root finding). Unless noted otherwise, you are expected to code up your own routines using the algorithms discussed in class and **cannot use special library functions** (except exp(), log(), etc.). Things like `numpy` arrays and extremely simple functions like `arange`, `linspace` or `hist` (without using advanced features) are OK. Things like matrix classes, factorials and gamma functions are not. Any type of algorithm discussed in class should of course always be coded by yourself (e.g. never use `np.interp` or `scipy.interpolate`). When in doubt, write your own, or ask us on the discussion boards.

You can use the routines you wrote for the tutorials, however, your routines must be written **by yourself** (and so **in no part** by someone else or an LLM). Codes will be checked for blatant copying (with other students as well as other sources), routines which are too similar get **zero points** at best.

For every main question you should write a **separate program**/separate code file(s). We **must** be able to run everything with **a single call to a script** `run.sh`[1], which downloads any data (data needed for an exercise **may not** be included in your code package but needs to be retrieved at runtime, see the `run.sh` example), runs your scripts and generates a PDF containing all your source code and outputs **in the following format**:

- Per main question, the code of any shared modules.
- Per sub-question, an explanation of what you did.
- Per sub-question, the code specific to it.
- Per sub-question, the output(s) along with discussion/captions.

Your code may be **handed in** however you'd like, for example by emailing a zip file or by sharing a github repository. If you organize your code in multiple folders, `run.sh` should be **in the top folder**. It's a good idea to have a fellow student **test** that your code works by running ./run.sh to make sure there are no permission errors or the like. Exercises that are not run with this single command or do not have their code and output in the PDF get **zero points** (this includes solutions in Jupyter notebooks).

Ensure (**test!**) that your code runs to completion on a Virtual Desktop machine (normal workstation)[2], using the **default** version and packages of `python3` (e.g. don't load modules only installed for you). Your code should have a total run time of **at most 10 minutes**, solutions generated will not be checked beyond this limit. All scripts should run to the end without user interference, so do not ask for input and **do not use plt.show()**, as it stops automatic execution.

For all routines you write, **explain** how they work in the comments of your code and **argue** your choices! Similarly, whenever your code outputs something **clearly indicate** next to the output/in the PDF what is being printed. This includes **discussing your plots in their captions**. If you used any sources outside of this course for writing your code, **give these sources explicitly**.

If a part of your code **does not run**, explain what you did so far and what the problem was and still include that part of the code in the PDF for possible partial credit. If you are unable to get a routine to work but you need it for a follow-up question, use a library routine in the follow-up (and clearly indicate that and why you do so).

Writing code that is very time- or memory-inefficient may incur a small **penalty** (e.g. using two matrices for an LU decomposition). Additional investigations/relevant plots may earn a small **bonus** (e.g. consistency/convergence checks).

Each sub-question starts with a reference to a relevant tutorial (T) and/or a reference to the relevant lecture (L). For example, *[L2,T2.3]* means the question is related to lecture 2 and question 3 in tutorial 2. Your previously written code for these will be a great starting point!

[1] Download **and adapt** the example here: `https://github.com/daalen/NURSolutionTemplate`.
[2] For more info, see `https://helpdesk.strw.leidenuniv.nl/wiki/doku.php?id=manuals:virtualdesktopserver`.

1. **Satellite galaxies around a massive central**
The spherical distribution of satellite galaxies around a central is often fitted with an NFW profile, under the assumption that it traces the dark matter halo mass. However, the following number density profile turns out to be a better match, from near the centre to well beyond the virial radius:

$$n(x) = A \langle N_{\rm sat} \rangle \left(\frac{x}{b}\right)^{a-3} \exp\left[-\left(\frac{x}{b}\right)^{c}\right]. \tag{1}$$

Here x is the radius relative to the virial radius, $x \equiv r/r_{\rm vir}$, and a, b and c are free parameters controlling the small-scale slope, transition scale and steepness of the exponential drop-off, respectively. A normalizes the profile such that the 3D spherical(!) integral from $x = 0$ to $x_{\rm max} = 5$ gives the average total number of satellites:

$$\iiint_V n(x)\,{\rm d}V = \langle N_{\rm sat} \rangle\,. \tag{2}$$

Throughout this exercise, take $a = 2.4$, $b = 0.25$, $c = 1.6$, $x_{\rm max} = 5$, $\langle N_{\rm sat} \rangle = 100$. To get you started, you can download a helpful Python script from `https://home.strw.leidenuniv.nl/~daalen/Handin_files/satellites.py`, including $n(x)$ and plotting routines.
Note that for this exercise, you will need to write your <u>own</u> random number generator, and that RNGs consisting <u>only</u> of (M)LCGs will get zero points.

(a) (4 points) *[L4,T4.1,T4.2]* Write a numerical integrator based on some form of Richardson extrapolation to solve equation (2) for A given those four parameters. Output A to full precision. *Hint: First think about what* ${\rm d}V$ *is for a spherical integral!*

(b) (6 points) *[L5,T5.2]* We want to generate 3D satellite positions such that they statistically follow the satellite profile in equation (1); that is, the probability distribution of the (relative) radii $x \in [0, 5)$ should be $p(x)\,{\rm d}x = N(x)\,{\rm d}x / \langle N_{\rm sat} \rangle$.[3] Use one of the methods discussed in class to sample this distribution. Generate 10,000 points and make a log-log plot showing $N(x)$ and a histogram of your 10,000 sampled points. Use 20 logarithmically-spaced bins between $x = 10^{-4}$ and $x = x_{\rm max}$ and don't forget to divide each bin by its width. Check if both profiles agree with each other (remember to correct for a normalization offset $10{,}000/\langle N_{\rm sat} \rangle = 100$). Make sure to set the vertical range such that the histogram and the analytical function can be compared well (*hint: the axis range in the plotting routine has already been set to good values if you do it correctly*).
Note: your random number generator must contain at least two different-method subgenerators (**test** the randomness of your total generator!), and make sure you understand the difference between a state and a seed (keep track of states, never reseed).

(c) (3 points) *[L6,T6.1]* Select 100 random satellite galaxies from (b) in a way that is guaranteed to:

1. select every galaxy with equal probability;
2. not draw the same galaxy twice;
3. not reject any draw.

Next sort the 100 drawn galaxies from smallest to largest radius and plot the number of galaxies within a radius, use a xlog plot from $x = 10^{-4}$ to $x = x_{\rm max}$.

(d) (6 points) *[L5,T5.1]* Numerically calculate ${\rm d}n(x)/{\rm d}x$ at $x = 1$. Output the value found alongside the analytical result, both to at least 12 significant digits. Choose your differentiation algorithm such that these are as close as possible.

(The hand-in continues on the next page.)

[3] Note that $n(x)$ is a number *density*, while $N(x)\,{\rm d}x$ is the *number* of galaxies in the infinitesimal range $[x, x + {\rm d}x)$. In other words, the probability to find a galaxy at some relative radius x is equal to the expected fraction of all satellites that reside at that radius, relative to the total number. How to go from $n(x)$ to $N(x)$ is part of the exercise.

2. **Heating and cooling in HII regions**
Let's consider an HII region similar to Orion. In an HII region we have primarily hot ionized gas that is heated by stars. In order to understand these regions in detail we need to calculate the equilibrium temperature. To calculate this we need to use root finders because the functions are too complicated to solve analytically. For this exercise you can choose the root finders that you prefer – however, whether you get full points depends on how fast the root is found (so be sure to print the number of steps and time taken). You can download Python implementations of the functions in this exercise from `https://home.strw.leidenuniv.nl/~daalen/Handin_files/heatingcooling.py`.

   (a) (5 points) *[L6,T6.2]* Lets consider an HII region that has only one heating and cooling mechanism. The heating is produced by photoionization given by:

$$\Gamma_{\rm pe} = \alpha_{\rm B} n_{\rm H} n_{\rm e} \psi k_{\rm B} T_{\rm c}. \tag{3}$$

   Here α_B is the case B recombination coefficient given by $2 \cdot 10^{-13}$ cm^3 s^{-1}, $n_{\rm H}$ and $n_{\rm e}$ are the hydrogen (or proton) and electron density, ψ is a numerical value close to one, which in our case is given by $\psi = 0.929$ and $T_{\rm c}$ is the stellar temperature (in our case $T_{\rm c} = 10^4\,{\rm K}$). The photoionization is balanced by the radiative recombination given by:

$$\Lambda_{\rm rr} = \alpha_{\rm B} n_{\rm H} n_{\rm e} \langle E_{\rm rr} \rangle, \tag{4}$$

   with:

$$\langle E_{\rm rr} \rangle = \left[0.684 - 0.0416 \ln \left(\frac{T_4}{Z^2} \right) \right] k_{\rm B} T. \tag{5}$$

   Here Z is the metallicity, which we assume to be solar ($Z = 0.015$), while $T_4 \equiv T/(10^4\,{\rm K})$. Calculate the equilibrium temperature in this case, i.e., the value of T at which equations (3) and (4) are equal, down to an accuracy of at least 0.1 K. For this, assume that the gas is fully ionized, i.e., $n_{\rm e} = n_{\rm H}$. Adopt the range $T = [1, 10^7]\,{\rm K}$ for the first bracket. If your method instead requires an initial guess, take the mid point of the given interval.

   (b) (3 points) *[L6,T6.2]* Let's now consider a more realistic configuration, with additional cooling by free-free emission ($\Lambda_{\rm FF}$), and heating by cosmic rays ($\Gamma_{\rm CR}$) and MHD waves ($\Gamma_{\rm MHD}$), as given by the following expressions:

$$\Lambda_{\rm FF} = 0.54\, T_4^{0.37} \alpha_{\rm B} n_{\rm e} n_{\rm H} k_{\rm B} T. \tag{6}$$

$$\Gamma_{\rm CR} = A n_{\rm e} \xi_{\rm CR}, \tag{7}$$

   in which A is a constant given by $A = 5 \cdot 10^{-10}$ erg, and $\xi_{\rm CR} = 10^{-15}$ s^{-1}, and

$$\Gamma_{\rm MHD} = 8.9 \cdot 10^{-26} n_{\rm H} T_4. \tag{8}$$

   Calculate the equilibrium temperature for a low, intermediate and high density gas with $n_{\rm e} = 10^{-4}$ cm^{-3}, 1 cm^{-3} and 10^4 cm^{-3}, respectively. Start from an interval $T = [1, 10^{15}]\,{\rm K}$, and aim for a relative target accuracy of 10^{-10} or better. What is the difference for the low, intermediate and high gas density?